UNIVERSITY OF WOLLONGONG IN DUBAI
Faculty of Engineering
and Information Sciences

Lab 3: Smart Warehouse & Cargo Logistics Controller

1D/2D Array Processing, Reshaping, Transposition, C-Structs & C-String Search

1. Introduction

1.1 Overview

In modern automated distribution centres, logistics controllers must manage thousands of cargo items in real-time. This lab models a Smart Warehouse & Cargo Logistics System that processes freight inventory using C's core structured data types. The application combines low-level array manipulation (for 1D storage streams and 2D storage grids) with high-level record management using C structures (struct) and string operations (string.h).

The program performs two main interconnected operational tasks:

- 1. Low-Level Freight Storage Engine (1D & 2D Array Transformations): Manages linear cargo tracking streams and transforms them into 2D physical storage grid layouts (Aisle × Shelf). It handles dynamic element deletion with left-shifting, insertion with right-shifting, duplicate tracking ID detection, array reversing (flipping), and matrix transpositions.
- 2. Cargo Inventory Record Manager (C-Structs & Strings): Maintains a structured database (struct CargoRecord) tracking freight items across 4 distribution zones. It handles interactive shipment registration, missing data handling (-1 sentinel values), automated total weight aggregation, status calculation (HD, D, C, P, F, undefined), and case-insensitive supplier/item searches using strcasecmp().

1.2 Student Learning Outcomes (SLOs)

- **1. 1D Array Manipulation:** Implement index-aligned element deletion (left-shifting), insertion (right-shifting), array reversal, and duplicate detection without out-of-bounds memory corruption.
- **2. 2D Matrix Transformations:** Copy 1D linear array streams into 2D grid matrices (row-by-row) and display transposed column-major matrix layouts.
- **3. Structured Data Types:** Define composite C structures (struct) storing scalar identifiers, strings, and nested float arrays.
- **4. String Processing & Search:** Utilise C-string library functions (string.h), specifically strcasecmp() for case-insensitive record searching.

2. Submission Guidelines

Students must submit the following items on Moodle before the deadline:

1. **C Source File**: Concisely commented warehouse_logistics.c file containing complete, compilable code.
2. **Lab Report consisting of:**
   A) Design Solution Document: Flowchart or Pseudocode for Question 1 array functions.

B) Testing Evidence: Terminal screenshots showing execution results for all array operations and kiosk tasks.

3. **In-Lab & Progress:** Weekly progress and live in-lab assessment via instructor observation and QA.

Lab session	Topic & Focus Area	In-Lab Tasks & Required Functions	Key Deliverables
Week 5 (Section 1)	**Low-Level Array Processing Operations**	• set_array: Populates array elements with index values. • rem_align: Deletes element at index pos by shifting remaining elements left. • insert_align: Inserts value at pos by shifting elements right • reshape: Maps 1D array into a 2D r × c grid matrix. • print_trans_matrix: Displays 2D grid column-by-column. • found_duplicate: Checks array for duplicate tracking IDs. • flip_array: Populates a new array with reversed elements.	1. Pre-Lab flowcharts or pseudocode for Q1 array algorithms. 2. Working C code for all 7 array processing functions verified inside main ().
Week 6 (Section 2)	**Cargo Structs, Strings & Data Validation**	• struct CargoRecord: Define structure with itemName, supplier, cargoID, zoneWeights[], totalWeight, and status • registerCargo(): Interactive registration, zone-by-zone weight input, and status grading assignment (HD, D, C, P, F, undefined) • searchUpdateCargo(): Search by ID or Supplier/Name using case-insensitive strcasecmp(), display attributes, and update fields. • topCargoItems(): Search and display all shipments holding the highest aggregate weight.	1. Implemented struct CargoRecord definition. 2. C code for registerCargo(), searchUpdateCargo(), and topCargoItems() with input validation.
Week 7	**Kiosk**	• Interactive Kiosk Menu:	**1.** Final compiled C source file

(Section 3)	**Integration, System Testing & In-Lab QA**	Combine Q1 array utilities and Q2 struct management into a menu loop. • Edge-Case Testing: Verify out-of-bounds array operations, missing zone weight entries, invalid search terms, and duplicate IDs. • Code Debugging	(warehouse_logistics.c). **2.** Design solution description document. **3.** Testing evidence screenshots. **4.** Live in-lab demonstrator QA sign-off

3. Theory & Key Equations

3.1 Low-Level Array Index Manipulation & Memory Shifting

Deletion with Left-Shifting (rem_align): When removing an element at index pos from a 1D array of length *N*, all subsequent elements from index pos + 1 up to *N - 1* must shift one position to the left. The active length of the array is then decremented by 1. *Formula:* arr[i] = arr[i + 1] for i = pos to *length - 2.

Operation	Array Memory Contents	Active Length
Initial Array	[10, 20, 30, 40, 50]	5
Delete arr[2] (val 30)	[10, 20, 40, 50, 50]	4 (Decremented)

Insertion with Right-Shifting (insert_align): Inserting a value at index pos requires shifting all elements from index *length - 1 down to pos one position to the right to create an empty slot. The new value is assigned to arr[pos] and length is incremented by 1. *Formula:* arr[i] = arr[i - 1] for i = *length down to pos + 1.

Operation	Array Memory Contents	Active Length
Initial Array	[10, 20, 30, 40, 50]	5
Insert 80 at pos=2	[10, 20, 80, 30, 40]	6 (Incremented)

3.2 Matrix Reshaping & Column-Major Transposition

Grid Reshaping (reshape): Linear 1D cargo data is copied into a 2D warehouse storage grid matrix grid[nRows][nCols] row by row. This mapping satisfies:
grid[i / nCols][i % nCols] = arr[i] for i = 0 to length - 1.
Precondition: The operation is valid if and only if length == nRows * nCols.

Column-Major Transposition (print_trans_matrix): To inspect warehouse aisles vertically, the 2D grid is printed column-by-column (each column printed on a new line). Outer loop iterates over columns j, inner loop over rows i.

3.3 C-Structs, Nested Arrays & C-Strings

In Question 2, each cargo record is defined using a C struct:

• itemName (string, e.g., 'Microprocessors') | • supplier (string, e.g., 'Intel')
• cargoID (integer) | • zoneWeights (1D float array across numOfZones)
• totalWeight (integer aggregate weight) | • status (string grade classification)

Case-Insensitive Searching: String comparisons for supplier or item name queries must use the standard C function strcasecmp(s1, s2) (from <strings.h> or <string.h>), returning 0 on match regardless of case.

3.4 Weight Classification & Status Criteria

When all zone weights are entered during registration or updates, the system sums the weight floats into totalWeight (rounded integer) and assigns a status classification grade as per Table 3.1. If any zone weight is skipped (set to -1), totalWeight is set to -1 and status to "undefined".

Aggregate Total Weight (kg)	Status Code	Cargo Category / Description
100 – 85 kg	**HD**	Heavy Duty Freight
84 – 75 kg	**D**	Deluxe Commercial Cargo
74 – 65 kg	**C**	Commercial Standard Cargo
64 – 50 kg	**P**	Standard Package
< 50 kg	**F**	Fragile / Light Weight
Missing Zone (-1)	**undefined**	Incomplete Inspection Record

4. Pre-Laboratory Tasks

Before writing code in VS Code, you must plan your algorithmic logic visually to build structured problem-solving skills:

Task 4.1: Flowchart / Pseudocode for Question 1 Array Functions (6 Marks):

Construct clear flowcharts or detailed pseudocode for the following low-level array manipulation functions:

1. rem_align(): Detailing index boundary validation, left-shifting loop logic, and pointer/length decrementation.
2. insert_align(): Detailing capacity validation, right-shifting loop logic, and element insertion.
3. reshape(): Detailing length validation (length == nRows * nCols) and 1D-to-2D mapping.
4. flip_array(): Detailing loop mechanics for reversing array elements into a target array.

5. Laboratory Tasks & Function Specifications

5.1 Question 1: Low-Level Array Processing (21 Marks)

Complete the C program warehouse_logistics.c by implementing the following array manipulation functions. **Constraint:** Do NOT use global variables or additional #define constants beyond those specified.

Function Prototype	Detailed Specification	Marks
void set_array(int arr[], int length)	Populates array elements relative to index: arr[i] = i for 0 <= i < length.	**2 Marks**
bool rem_align(int arr[], int *length, int pos)	Deletes element at index pos. Validates 0 <= pos < *length. Shifts elements left (arr[i] = arr[i+1]), decrements *length, and returns true. Returns false if invalid.	**4 Marks**
bool insert_alig n(int arr[], int *length, int pos, int value, int maxCap)	Inserts value at index pos. Validates 0 <= pos <= *length and *length < maxCap. Shifts elements right (arr[i+1] = arr[i]), inserts value, increments *length, and returns true.	**4 Marks**
bool reshape(int arr[], int length, int r, int c, int arr2d[r][c])	Maps 1D array into 2D grid matrix (r × c). Validates length == r * c. Populates arr2d[i/c][i%c] = arr[i] and returns true.	**4 Marks**
print_trans _matrix(int r, int c, int arr2d[r][c])	Displays 2D matrix column-by-column (aisle layout): outer loop over columns 0..c-1, inner loop over rows 0..r-1.	**2 Marks**
bool found_duplic ate(int arr[], int length)	Scans array using nested loops. Returns true if any duplicate tracking ID exists; otherwise false.	**4 Marks**
bool flip_array(int arr[], int length, int flipped[])	Populates flipped[] with reversed content of arr[] (flipped[i] = arr[length-1-i]). Returns true.	**5 Marks**

5.2 Question 2: Warehouse Cargo Management System (21 Marks)

Design and implement a C program that maintains and processes records for a list of warehouse cargo shipments. Assume all shipments have unique names and suppliers. Define constants numOfZones (number of inspection zones) and numOfItems (number of cargo records in class/list).

Define struct CargoRecord containing attributes: itemName (string), supplier (string), cargoID (int), zoneWeights (float array of size numOfZones), totalWeight (int), and status (string). Program must validate numerical inputs before processing.

Implement a menu-driven interface with the following 3 primary functions + exit option:

1. registerCargo [6 Marks]

void registerCargo(CargoRecord database[], int *count, int maxCap)

Prompts user to input cargoID. Then, for each zone (Zone 0, Zone 1, ..., Zone numOfZones-1), asks if user wants to input weight. If user answers 'Yes', prompts and updates zoneWeights[i]. If 'No', sets zoneWeights[i] = -1. If ALL zone weights were input, calculates totalWeight (rounded sum) and assigns status string as per Table 3.1. Otherwise sets totalWeight = -1 and status = "undefined".

2. searchUpdateCargo [6 Marks]

void searchUpdateCargo(CargoRecord database[], int count)

Prompts user to search by cargoID or supplier/itemName. Performs case-insensitive matching using strcasecmp(). If no match found, prints "unfound". Otherwise displays matching record attributes. Then prompts user if they want to update any of the first four attributes (itemName, supplier, cargoID, zoneWeights) and updates record. Recalculates totalWeight and status when applicable.

3. topCargoItems [3 Marks]

void topCargoItems(CargoRecord database[], int count)

Scans the record array and prints full details of all cargo items holding the highest totalWeight in the system.

6. Assessment Tasks & Grading Rubric

Assessment Component	Evaluation Criteria & Deliverables	Marks
Q1: Array Processing C Code	Correct implementation of set_array, rem_align, insert_align, reshape, print_trans_matrix, found_duplicate, and flip_array.	**21**
Q2: Cargo System C Code	Correct implementation of struct CargoRecord, registerCargo, searchUpdateCargo (strcasecmp), topCargoItems, and menu loop.	**21**
Design Solution Document	Flowchart or Pseudocode for Question 1 array processing functions.	**6**
Testing Evidence	Terminal execution screenshots with discussion verifying all Q1 & Q2 operations.	**6**
Data Input Validation	Robust handling of invalid menu inputs, invalid indices, and invalid weights.	**2**
Main Loop & Execution Structure	Clean main() function, menu driving loop, memory management, and code formatting.	**4**

In-Lab Assessment & QA	Weekly progress in class, instructor observation, and live VS Code debugging QA.	**14**
Lab Report	Completed report with all the tasks, screenshot samples and flowcharts, etc.	**30**
TOTAL GRADE		**100 Marks**

Appendix A: Sample Terminal Run Transcripts

The following terminal run transcripts demonstrate the expected behaviour of the completed software system.

A.1 Part 2 Cargo Registration & Status Calculation (`registerCargo`)

```
--- CARGO REGISTRATION (ZONE WEIGHTS ENTRY) ---
Enter [redacted] / Cargo Item ID to register: 101
Selected Cargo: ID=101, Name=Cargo_1, Supplier=Supplier_A
Do you want to input weight for Zone 0? (1 = Yes, 0 = No): 1
Enter weight for Zone 0 (kg): 25.5
Do you want to input weight for Zone 1? (1 = Yes, 0 = No): 1
Enter weight for Zone 1 (kg): 20.0
Do you want to input weight for Zone 2? (1 = Yes, 0 = No): 1
Enter weight for Zone 2 (kg): 22.0
Do you want to input weight for Zone 3? (1 = Yes, 0 = No): 1
Enter weight for Zone 3 (kg): 18.5
[SUCCESS] Updated Record: Total Weight = 86 kg, Status = HD
```

A.2 Part 2 Case-Insensitive Search & Record Update (`searchUpdateCargo`)

```
--- SEARCH & UPDATE CARGO RECORDS ---
Search by: 1. Item ID   2. Supplier / Item Name
Select choice (1-2): 2
Enter Supplier or Item Name (case-insensitive): supplier_a

[MATCH FOUND] ItemID: 101 | Name: Cargo_1 | Supplier: Supplier_A | Total Weight: 86 kg | Status: HD

Do you want to update this record? (1 = Yes, 0 = No): 1
Enter new Item Name: HeavyBox_A
Enter new Supplier Name: ApexLogistics
Enter new Item ID: 201
[SUCCESS] Record updated successfully.
```

A.3 Full Freight Database Display (`displayAllCargo`)

```
================================================================================
                    CURRENT WAREHOUSE FREIGHT DATABASE
================================================================================
  ID    | Item Name    | Supplier     | Total Wt | Status    | Zone Weights
--------------------------------------------------------------------------------
  201   | HeavyBox_A   | ApexLogist   | 86       | HD        | [ 25.5 20.0 22.0 18.5 ]
  102   | Cargo_2      | Supplier_B   | -1       | undefined | [ N/A N/A N/A N/A ]
  103   | Cargo_3      | Supplier_C   | -1       | undefined | [ N/A N/A N/A N/A ]
  104   | Cargo_4      | Supplier_D   | -1       | undefined | [ N/A N/A N/A N/A ]
  105   | Cargo_5      | Supplier_E   | -1       | undefined | [ N/A N/A N/A N/A ]
================================================================================
```

END OF LAB 3 HANDOUT